0ngewvkll

September 21, 2025

1 Week-4 Follow Along

DS-532 Shivali Mate

1.1 Connect to MySQL database (Sakila)

[5]:
```
!pip install mysql-connector-python
```

```
Requirement already satisfied: mysql-connector-python in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages (9.4.0)
```

[11]:
```
config = {
  'user': 'root',
  'password': 'root',
  'host': '127.0.0.1',
  #'port': 8889,
  'database': 'sakila',
  'raise_on_warnings': True
}
```

[12]:
```
# create a MySQL server connection object
import mysql.connector
mydb = mysql.connector.connect(**config)
```

[13]:
```
# cursor object
my_cursor = mydb.cursor(dictionary=True)
```

1.1.1 Information Schema

Reading: Schema Tables

- `information_schema.tables`
  – storing information about all the tables in that particular schema
- `information_schema.column`
  – storing information about all the columns in that particular table
- `information_schema.key_column_usage`
  – storing information about all the keys and indexes in that particular schema
- `information_schema.table_constraints`

– storing information about all the constraints in all the tables in that particular schema

[19]:
```
my_cursor.execute(" SELECT table_name as table_name FROM information_schema.
 ↪tables WHERE table_schema ='sakila' ")
my_result = my_cursor.fetchall()
```

[20]:
```
for table in my_result:
    print(table)
```

```
{'table_name': 'actor'}
{'table_name': 'actor_info'}
{'table_name': 'address'}
{'table_name': 'category'}
{'table_name': 'city'}
{'table_name': 'country'}
{'table_name': 'customer'}
{'table_name': 'customer_list'}
{'table_name': 'film'}
{'table_name': 'film_actor'}
{'table_name': 'film_category'}
{'table_name': 'film_list'}
{'table_name': 'film_text'}
{'table_name': 'inventory'}
{'table_name': 'language'}
{'table_name': 'nicer_but_slower_film_list'}
{'table_name': 'payment'}
{'table_name': 'rental'}
{'table_name': 'sales_by_film_category'}
{'table_name': 'sales_by_store'}
{'table_name': 'staff'}
{'table_name': 'staff_list'}
{'table_name': 'store'}
```

[21]:
```
# Just fetching the tables
print(f"Table: {my_result[0]['table_name']}")
print(f"Total number of tables: {len(my_result)}")
```

```
Table: actor
Total number of tables: 23
```

[22]:
```
for table_num in range(len(my_result)):
    table_name = my_result[table_num]['table_name']
    my_cursor.execute(f" SELECT column_name, data_type, ordinal_position FROM␣
 ↪information_schema.columns WHERE table_schema ='sakila' AND␣
 ↪table_name='{table_name}' ")
    column_names = my_cursor.fetchall()
    print()
    print(f'column names for table: {table_name}')
```

```
    print('-------------------------------------')
    print(column_names)
```

```
column names for table: actor
-------------------------------------
[{'COLUMN_NAME': 'actor_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'first_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'last_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 4}]

column names for table: actor_info
-------------------------------------
[{'COLUMN_NAME': 'actor_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'first_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'last_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'film_info', 'DATA_TYPE': 'text', 'ORDINAL_POSITION': 4}]

column names for table: address
-------------------------------------
[{'COLUMN_NAME': 'address_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'address', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'address2', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'district', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'city_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'postal_code', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'phone', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 7},
{'COLUMN_NAME': 'location', 'DATA_TYPE': 'geometry', 'ORDINAL_POSITION': 8},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 9}]

column names for table: category
-------------------------------------
[{'COLUMN_NAME': 'category_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 3}]

column names for table: city
-------------------------------------
[{'COLUMN_NAME': 'city_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'city', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'country_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 4}]

column names for table: country
-------------------------------------
[{'COLUMN_NAME': 'country_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'country', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
```

{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 3}]

column names for table: customer

[{'COLUMN_NAME': 'customer_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'store_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'first_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'last_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'email', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'address_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'active', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 7},
{'COLUMN_NAME': 'create_date', 'DATA_TYPE': 'datetime', 'ORDINAL_POSITION': 8},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 9}]

column names for table: customer_list

[{'COLUMN_NAME': 'ID', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'address', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'zip code', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'phone', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'city', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'country', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 7},
{'COLUMN_NAME': 'notes', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 8},
{'COLUMN_NAME': 'SID', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 9}]

column names for table: film

[{'COLUMN_NAME': 'film_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'title', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'description', 'DATA_TYPE': 'text', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'release_year', 'DATA_TYPE': 'year', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'language_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'original_language_id', 'DATA_TYPE': 'tinyint',
'ORDINAL_POSITION': 6}, {'COLUMN_NAME': 'rental_duration', 'DATA_TYPE':
'tinyint', 'ORDINAL_POSITION': 7}, {'COLUMN_NAME': 'rental_rate', 'DATA_TYPE':
'decimal', 'ORDINAL_POSITION': 8}, {'COLUMN_NAME': 'length', 'DATA_TYPE':
'smallint', 'ORDINAL_POSITION': 9}, {'COLUMN_NAME': 'replacement_cost',
'DATA_TYPE': 'decimal', 'ORDINAL_POSITION': 10}, {'COLUMN_NAME': 'rating',
'DATA_TYPE': 'enum', 'ORDINAL_POSITION': 11}, {'COLUMN_NAME':
'special_features', 'DATA_TYPE': 'set', 'ORDINAL_POSITION': 12}, {'COLUMN_NAME':
'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 13}]

column names for table: film_actor

[{'COLUMN_NAME': 'actor_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'film_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 3}]

```
column names for table: film_category
-------------------------------------
[{'COLUMN_NAME': 'film_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'category_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 3}]

column names for table: film_list
-------------------------------------
[{'COLUMN_NAME': 'FID', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'title', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'description', 'DATA_TYPE': 'text', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'category', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'price', 'DATA_TYPE': 'decimal', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'length', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'rating', 'DATA_TYPE': 'enum', 'ORDINAL_POSITION': 7},
{'COLUMN_NAME': 'actors', 'DATA_TYPE': 'text', 'ORDINAL_POSITION': 8}]

column names for table: film_text
-------------------------------------
[{'COLUMN_NAME': 'film_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'title', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'description', 'DATA_TYPE': 'text', 'ORDINAL_POSITION': 3}]

column names for table: inventory
-------------------------------------
[{'COLUMN_NAME': 'inventory_id', 'DATA_TYPE': 'mediumint', 'ORDINAL_POSITION':
1}, {'COLUMN_NAME': 'film_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'store_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 4}]

column names for table: language
-------------------------------------
[{'COLUMN_NAME': 'language_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'name', 'DATA_TYPE': 'char', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 3}]

column names for table: nicer_but_slower_film_list
-------------------------------------
[{'COLUMN_NAME': 'FID', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'title', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'description', 'DATA_TYPE': 'text', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'category', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'price', 'DATA_TYPE': 'decimal', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'length', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'rating', 'DATA_TYPE': 'enum', 'ORDINAL_POSITION': 7},
{'COLUMN_NAME': 'actors', 'DATA_TYPE': 'text', 'ORDINAL_POSITION': 8}]
```

```
column names for table: payment
-----------------------------------
[{'COLUMN_NAME': 'payment_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'customer_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'staff_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'rental_id', 'DATA_TYPE': 'int', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'amount', 'DATA_TYPE': 'decimal', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'payment_date', 'DATA_TYPE': 'datetime', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 7}]

column names for table: rental
-----------------------------------
[{'COLUMN_NAME': 'rental_id', 'DATA_TYPE': 'int', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'rental_date', 'DATA_TYPE': 'datetime', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'inventory_id', 'DATA_TYPE': 'mediumint', 'ORDINAL_POSITION':
3}, {'COLUMN_NAME': 'customer_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION':
4}, {'COLUMN_NAME': 'return_date', 'DATA_TYPE': 'datetime', 'ORDINAL_POSITION':
5}, {'COLUMN_NAME': 'staff_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION': 7}]

column names for table: sales_by_film_category
-----------------------------------
[{'COLUMN_NAME': 'category', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'total_sales', 'DATA_TYPE': 'decimal', 'ORDINAL_POSITION': 2}]

column names for table: sales_by_store
-----------------------------------
[{'COLUMN_NAME': 'store', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'manager', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'total_sales', 'DATA_TYPE': 'decimal', 'ORDINAL_POSITION': 3}]

column names for table: staff
-----------------------------------
[{'COLUMN_NAME': 'staff_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'first_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'last_name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'address_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'picture', 'DATA_TYPE': 'blob', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'email', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'store_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 7},
{'COLUMN_NAME': 'active', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 8},
{'COLUMN_NAME': 'username', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 9},
{'COLUMN_NAME': 'password', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 10},
{'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION':
11}]

column names for table: staff_list
-----------------------------------
```

```
[{'COLUMN_NAME': 'ID', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'name', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 2},
{'COLUMN_NAME': 'address', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 3},
{'COLUMN_NAME': 'zip code', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 4},
{'COLUMN_NAME': 'phone', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 5},
{'COLUMN_NAME': 'city', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 6},
{'COLUMN_NAME': 'country', 'DATA_TYPE': 'varchar', 'ORDINAL_POSITION': 7},
{'COLUMN_NAME': 'SID', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 8}]

column names for table: store
-------------------------------------
[{'COLUMN_NAME': 'store_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION': 1},
{'COLUMN_NAME': 'manager_staff_id', 'DATA_TYPE': 'tinyint', 'ORDINAL_POSITION':
2}, {'COLUMN_NAME': 'address_id', 'DATA_TYPE': 'smallint', 'ORDINAL_POSITION':
3}, {'COLUMN_NAME': 'last_update', 'DATA_TYPE': 'timestamp', 'ORDINAL_POSITION':
4}]
```

[23]:
```
my_cursor.execute("show index from actor;")
my_index= my_cursor.fetchall()
for idx in my_index:
    print(idx)
```

```
{'Table': 'actor', 'Non_unique': 0, 'Key_name': 'PRIMARY', 'Seq_in_index': 1,
'Column_name': 'actor_id', 'Collation': 'A', 'Cardinality': 200, 'Sub_part':
None, 'Packed': None, 'Null': '', 'Index_type': 'BTREE', 'Comment': '',
'Index_comment': '', 'Visible': 'YES', 'Expression': None}
{'Table': 'actor', 'Non_unique': 1, 'Key_name': 'idx_actor_last_name',
'Seq_in_index': 1, 'Column_name': 'last_name', 'Collation': 'A', 'Cardinality':
121, 'Sub_part': None, 'Packed': None, 'Null': '', 'Index_type': 'BTREE',
'Comment': '', 'Index_comment': '', 'Visible': 'YES', 'Expression': None}
```

[24]:
```
# constraints
sql_query='''SELECT a.table_name,
       a.constraint_name,
       b.column_name,
       a.constraint_type
FROM information_schema.table_constraints AS a,
     information_schema.key_column_usage AS b
WHERE a.table_name = 'address'
AND a.table_schema = 'sakila'
and a.table_name = b.table_name
AND a.table_schema = b.table_schema
AND a.constraint_name = b.constraint_name;
'''

my_cursor.execute(sql_query)
constraints=my_cursor.fetchall()
```

```
for row in constraints:
    print(row)
```

```
{'TABLE_NAME': 'address', 'CONSTRAINT_NAME': 'PRIMARY', 'COLUMN_NAME':
'address_id', 'CONSTRAINT_TYPE': 'PRIMARY KEY'}
{'TABLE_NAME': 'address', 'CONSTRAINT_NAME': 'fk_address_city', 'COLUMN_NAME':
'city_id', 'CONSTRAINT_TYPE': 'FOREIGN KEY'}
```

[25]:
```
# Foreign keys
sql_query='''SELECT * FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'sakila' AND TABLE_NAME = 'address';
'''

my_cursor.execute(sql_query)
foreign_keys=my_cursor.fetchall()
for row in foreign_keys:
    print(row)
```

```
{'CONSTRAINT_CATALOG': 'def', 'CONSTRAINT_SCHEMA': 'sakila', 'CONSTRAINT_NAME':
'PRIMARY', 'TABLE_CATALOG': 'def', 'TABLE_SCHEMA': 'sakila', 'TABLE_NAME':
'address', 'COLUMN_NAME': 'address_id', 'ORDINAL_POSITION': 1,
'POSITION_IN_UNIQUE_CONSTRAINT': None, 'REFERENCED_TABLE_SCHEMA': None,
'REFERENCED_TABLE_NAME': None, 'REFERENCED_COLUMN_NAME': None}
{'CONSTRAINT_CATALOG': 'def', 'CONSTRAINT_SCHEMA': 'sakila', 'CONSTRAINT_NAME':
'fk_address_city', 'TABLE_CATALOG': 'def', 'TABLE_SCHEMA': 'sakila',
'TABLE_NAME': 'address', 'COLUMN_NAME': 'city_id', 'ORDINAL_POSITION': 1,
'POSITION_IN_UNIQUE_CONSTRAINT': 1, 'REFERENCED_TABLE_SCHEMA': 'sakila',
'REFERENCED_TABLE_NAME': 'city', 'REFERENCED_COLUMN_NAME': 'city_id'}
```

[26]:
```
sql_query='''SHOW INDEX FROM sakila.address;
'''

my_cursor.execute(sql_query)
foreign_index=my_cursor.fetchall()
for row in foreign_index:
    print(row)
```

```
{'Table': 'address', 'Non_unique': 0, 'Key_name': 'PRIMARY', 'Seq_in_index': 1,
'Column_name': 'address_id', 'Collation': 'A', 'Cardinality': 603, 'Sub_part':
None, 'Packed': None, 'Null': '', 'Index_type': 'BTREE', 'Comment': '',
'Index_comment': '', 'Visible': 'YES', 'Expression': None}
{'Table': 'address', 'Non_unique': 1, 'Key_name': 'idx_fk_city_id',
'Seq_in_index': 1, 'Column_name': 'city_id', 'Collation': 'A', 'Cardinality':
599, 'Sub_part': None, 'Packed': None, 'Null': '', 'Index_type': 'BTREE',
'Comment': '', 'Index_comment': '', 'Visible': 'YES', 'Expression': None}
{'Table': 'address', 'Non_unique': 1, 'Key_name': 'idx_location',
'Seq_in_index': 1, 'Column_name': 'location', 'Collation': 'A', 'Cardinality':
603, 'Sub_part': 32, 'Packed': None, 'Null': '', 'Index_type': 'SPATIAL',
```

```
'Comment': '', 'Index_comment': '', 'Visible': 'YES', 'Expression': None}
```

Here, we can see that there is an index present on the foreign key on the column city_id. In MySQL, when you define a foreign key constraint, an index on the columns involved is automatically created if it does not exist.

[27]:
```
sql_query='''SELECT CONCAT('SELECT COUNT(*) FROM ', table_schema, '.',␣
 ↪table_name, ';')
FROM information_schema.tables
WHERE table_schema = 'sakila';
'''

my_cursor.execute(sql_query)
cnt=my_cursor.fetchall()
for row in cnt:
    print(list(row.values())[0])
```

```
SELECT COUNT(*) FROM sakila.actor;
SELECT COUNT(*) FROM sakila.actor_info;
SELECT COUNT(*) FROM sakila.address;
SELECT COUNT(*) FROM sakila.category;
SELECT COUNT(*) FROM sakila.city;
SELECT COUNT(*) FROM sakila.country;
SELECT COUNT(*) FROM sakila.customer;
SELECT COUNT(*) FROM sakila.customer_list;
SELECT COUNT(*) FROM sakila.film;
SELECT COUNT(*) FROM sakila.film_actor;
SELECT COUNT(*) FROM sakila.film_category;
SELECT COUNT(*) FROM sakila.film_list;
SELECT COUNT(*) FROM sakila.film_text;
SELECT COUNT(*) FROM sakila.inventory;
SELECT COUNT(*) FROM sakila.language;
SELECT COUNT(*) FROM sakila.nicer_but_slower_film_list;
SELECT COUNT(*) FROM sakila.payment;
SELECT COUNT(*) FROM sakila.rental;
SELECT COUNT(*) FROM sakila.sales_by_film_category;
SELECT COUNT(*) FROM sakila.sales_by_store;
SELECT COUNT(*) FROM sakila.staff;
SELECT COUNT(*) FROM sakila.staff_list;
SELECT COUNT(*) FROM sakila.store;
```

[28]:
```
sql_query='''SELECT CONCAT('ALTER TABLE ', table_name,' DROP FOREIGN KEY ',␣
 ↪constraint_name, ';')
FROM information_schema.KEY_COLUMN_USAGE
WHERE table_schema = 'sakila' AND REFERENCED_TABLE_NAME IS NOT NULL;
'''

my_cursor.execute(sql_query)
```

```
cnt=my_cursor.fetchall()
for row in cnt:
    print(list(row.values())[0])
```

```
ALTER TABLE address DROP FOREIGN KEY fk_address_city;
ALTER TABLE city DROP FOREIGN KEY fk_city_country;
ALTER TABLE customer DROP FOREIGN KEY fk_customer_address;
ALTER TABLE customer DROP FOREIGN KEY fk_customer_store;
ALTER TABLE film DROP FOREIGN KEY fk_film_language;
ALTER TABLE film DROP FOREIGN KEY fk_film_language_original;
ALTER TABLE film_actor DROP FOREIGN KEY fk_film_actor_actor;
ALTER TABLE film_actor DROP FOREIGN KEY fk_film_actor_film;
ALTER TABLE film_category DROP FOREIGN KEY fk_film_category_category;
ALTER TABLE film_category DROP FOREIGN KEY fk_film_category_film;
ALTER TABLE inventory DROP FOREIGN KEY fk_inventory_film;
ALTER TABLE inventory DROP FOREIGN KEY fk_inventory_store;
ALTER TABLE payment DROP FOREIGN KEY fk_payment_customer;
ALTER TABLE payment DROP FOREIGN KEY fk_payment_rental;
ALTER TABLE payment DROP FOREIGN KEY fk_payment_staff;
ALTER TABLE rental DROP FOREIGN KEY fk_rental_customer;
ALTER TABLE rental DROP FOREIGN KEY fk_rental_inventory;
ALTER TABLE rental DROP FOREIGN KEY fk_rental_staff;
ALTER TABLE staff DROP FOREIGN KEY fk_staff_address;
ALTER TABLE staff DROP FOREIGN KEY fk_staff_store;
ALTER TABLE store DROP FOREIGN KEY fk_store_address;
ALTER TABLE store DROP FOREIGN KEY fk_store_staff;
```

You check the foreign key constraints are present in all the tables in the database

[29]:
```
# OPTIONAL
sql_query='''SELECT
  table_name,
  column_name,
  constraint_name,
  referenced_table_name,
  referenced_column_name
FROM
  information_schema.KEY_COLUMN_USAGE
WHERE
  REFERENCED_TABLE_SCHEMA = 'sakila'
  AND REFERENCED_TABLE_NAME IS NOT NULL;
'''


my_cursor.execute(sql_query)
foreign_keys=my_cursor.fetchall()
for row in foreign_keys:
    print(row)
```

{'TABLE_NAME': 'address', 'COLUMN_NAME': 'city_id', 'CONSTRAINT_NAME':
'fk_address_city', 'REFERENCED_TABLE_NAME': 'city', 'REFERENCED_COLUMN_NAME':
'city_id'}
{'TABLE_NAME': 'city', 'COLUMN_NAME': 'country_id', 'CONSTRAINT_NAME':
'fk_city_country', 'REFERENCED_TABLE_NAME': 'country', 'REFERENCED_COLUMN_NAME':
'country_id'}
{'TABLE_NAME': 'customer', 'COLUMN_NAME': 'address_id', 'CONSTRAINT_NAME':
'fk_customer_address', 'REFERENCED_TABLE_NAME': 'address',
'REFERENCED_COLUMN_NAME': 'address_id'}
{'TABLE_NAME': 'customer', 'COLUMN_NAME': 'store_id', 'CONSTRAINT_NAME':
'fk_customer_store', 'REFERENCED_TABLE_NAME': 'store', 'REFERENCED_COLUMN_NAME':
'store_id'}
{'TABLE_NAME': 'film', 'COLUMN_NAME': 'language_id', 'CONSTRAINT_NAME':
'fk_film_language', 'REFERENCED_TABLE_NAME': 'language',
'REFERENCED_COLUMN_NAME': 'language_id'}
{'TABLE_NAME': 'film', 'COLUMN_NAME': 'original_language_id', 'CONSTRAINT_NAME':
'fk_film_language_original', 'REFERENCED_TABLE_NAME': 'language',
'REFERENCED_COLUMN_NAME': 'language_id'}
{'TABLE_NAME': 'film_actor', 'COLUMN_NAME': 'actor_id', 'CONSTRAINT_NAME':
'fk_film_actor_actor', 'REFERENCED_TABLE_NAME': 'actor',
'REFERENCED_COLUMN_NAME': 'actor_id'}
{'TABLE_NAME': 'film_actor', 'COLUMN_NAME': 'film_id', 'CONSTRAINT_NAME':
'fk_film_actor_film', 'REFERENCED_TABLE_NAME': 'film', 'REFERENCED_COLUMN_NAME':
'film_id'}
{'TABLE_NAME': 'film_category', 'COLUMN_NAME': 'category_id', 'CONSTRAINT_NAME':
'fk_film_category_category', 'REFERENCED_TABLE_NAME': 'category',
'REFERENCED_COLUMN_NAME': 'category_id'}
{'TABLE_NAME': 'film_category', 'COLUMN_NAME': 'film_id', 'CONSTRAINT_NAME':
'fk_film_category_film', 'REFERENCED_TABLE_NAME': 'film',
'REFERENCED_COLUMN_NAME': 'film_id'}
{'TABLE_NAME': 'inventory', 'COLUMN_NAME': 'film_id', 'CONSTRAINT_NAME':
'fk_inventory_film', 'REFERENCED_TABLE_NAME': 'film', 'REFERENCED_COLUMN_NAME':
'film_id'}
{'TABLE_NAME': 'inventory', 'COLUMN_NAME': 'store_id', 'CONSTRAINT_NAME':
'fk_inventory_store', 'REFERENCED_TABLE_NAME': 'store',
'REFERENCED_COLUMN_NAME': 'store_id'}
{'TABLE_NAME': 'payment', 'COLUMN_NAME': 'customer_id', 'CONSTRAINT_NAME':
'fk_payment_customer', 'REFERENCED_TABLE_NAME': 'customer',
'REFERENCED_COLUMN_NAME': 'customer_id'}
{'TABLE_NAME': 'payment', 'COLUMN_NAME': 'rental_id', 'CONSTRAINT_NAME':
'fk_payment_rental', 'REFERENCED_TABLE_NAME': 'rental',
'REFERENCED_COLUMN_NAME': 'rental_id'}
{'TABLE_NAME': 'payment', 'COLUMN_NAME': 'staff_id', 'CONSTRAINT_NAME':
'fk_payment_staff', 'REFERENCED_TABLE_NAME': 'staff', 'REFERENCED_COLUMN_NAME':
'staff_id'}
{'TABLE_NAME': 'rental', 'COLUMN_NAME': 'customer_id', 'CONSTRAINT_NAME':
'fk_rental_customer', 'REFERENCED_TABLE_NAME': 'customer',
'REFERENCED_COLUMN_NAME': 'customer_id'}

```
{'TABLE_NAME': 'rental', 'COLUMN_NAME': 'inventory_id', 'CONSTRAINT_NAME':
'fk_rental_inventory', 'REFERENCED_TABLE_NAME': 'inventory',
'REFERENCED_COLUMN_NAME': 'inventory_id'}
{'TABLE_NAME': 'rental', 'COLUMN_NAME': 'staff_id', 'CONSTRAINT_NAME':
'fk_rental_staff', 'REFERENCED_TABLE_NAME': 'staff', 'REFERENCED_COLUMN_NAME':
'staff_id'}
{'TABLE_NAME': 'staff', 'COLUMN_NAME': 'address_id', 'CONSTRAINT_NAME':
'fk_staff_address', 'REFERENCED_TABLE_NAME': 'address',
'REFERENCED_COLUMN_NAME': 'address_id'}
{'TABLE_NAME': 'staff', 'COLUMN_NAME': 'store_id', 'CONSTRAINT_NAME':
'fk_staff_store', 'REFERENCED_TABLE_NAME': 'store', 'REFERENCED_COLUMN_NAME':
'store_id'}
{'TABLE_NAME': 'store', 'COLUMN_NAME': 'address_id', 'CONSTRAINT_NAME':
'fk_store_address', 'REFERENCED_TABLE_NAME': 'address',
'REFERENCED_COLUMN_NAME': 'address_id'}
{'TABLE_NAME': 'store', 'COLUMN_NAME': 'manager_staff_id', 'CONSTRAINT_NAME':
'fk_store_staff', 'REFERENCED_TABLE_NAME': 'staff', 'REFERENCED_COLUMN_NAME':
'staff_id'}
```

[30]:
```
sql_query='''SELECT CONCAT('INSERT INTO actor(actor_id, first_name, last_name)␣
 ↪VALUES(', actor_id, ', "', first_name, '", "', last_name, '");')
FROM actor
WHERE actor_id = 10;
'''

my_cursor.execute(sql_query)
cnt=my_cursor.fetchall()
for row in cnt:
    print(list(row.values())[0])
```

```
INSERT INTO actor(actor_id, first_name, last_name) VALUES(10, "CHRISTIAN",
"GABLE");
```

[31]:
```
##Query to list data dictionary views and their purposes:
sql_query='''SELECT table_name, table_comment
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'sakila';

'''



my_cursor.execute(sql_query)
views=my_cursor.fetchall()
for row in views:
    print(row)
```

```
{'TABLE_NAME': 'actor', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'actor_info', 'TABLE_COMMENT': 'VIEW'}
```

```
{'TABLE_NAME': 'address', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'category', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'city', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'country', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'customer', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'customer_list', 'TABLE_COMMENT': 'VIEW'}
{'TABLE_NAME': 'film', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'film_actor', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'film_category', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'film_list', 'TABLE_COMMENT': 'VIEW'}
{'TABLE_NAME': 'film_text', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'inventory', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'language', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'nicer_but_slower_film_list', 'TABLE_COMMENT': 'VIEW'}
{'TABLE_NAME': 'payment', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'rental', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'sales_by_film_category', 'TABLE_COMMENT': 'VIEW'}
{'TABLE_NAME': 'sales_by_store', 'TABLE_COMMENT': 'VIEW'}
{'TABLE_NAME': 'staff', 'TABLE_COMMENT': ''}
{'TABLE_NAME': 'staff_list', 'TABLE_COMMENT': 'VIEW'}
{'TABLE_NAME': 'store', 'TABLE_COMMENT': ''}
```

[32]:
```
##Query to describe the columns in a given data dictionary view:
sql_query='''SELECT COLUMN_NAME, COLUMN_COMMENT
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'sakila' AND TABLE_NAME = 'actor';


'''


my_cursor.execute(sql_query)
views=my_cursor.fetchall()
for row in views:
    print(row)
```

```
{'COLUMN_NAME': 'actor_id', 'COLUMN_COMMENT': ''}
{'COLUMN_NAME': 'first_name', 'COLUMN_COMMENT': ''}
{'COLUMN_NAME': 'last_name', 'COLUMN_COMMENT': ''}
{'COLUMN_NAME': 'last_update', 'COLUMN_COMMENT': ''}
```

1.2 Strings in SQL

Make sure to use a different connection and different cursor object

[40]:
```
shivali_config = {
  'user': 'root',
  'password': 'root',
```

```
  'host': '127.0.0.1',
  # 'port': 8889,
  'database': 'shivali',
  'raise_on_warnings': True
}
```

[41]:
```
shivali_db = mysql.connector.connect(**shivali_config)
# cursor object
new_cursor = shivali_db.cursor(dictionary=True)
```

[42]:
```
new_cursor.execute('''SELECT * FROM emp''')
for emp_table in new_cursor:
    print(emp_table)
```

```
{'EMPNO': 7369, 'ENAME': 'SMITH', 'JOB': 'CLERK', 'MGR': '7902', 'HIREDATE':
'17-Dec-05', 'SAL': 800, 'COMM': '', 'DEPTNO': 20}
{'EMPNO': 7499, 'ENAME': 'ALLEN', 'JOB': 'SALESMAN', 'MGR': '7698', 'HIREDATE':
'20-Feb-06', 'SAL': 1600, 'COMM': '300', 'DEPTNO': 30}
{'EMPNO': 7521, 'ENAME': 'WARD', 'JOB': 'SALESMAN', 'MGR': '7698', 'HIREDATE':
'22-Feb-06', 'SAL': 1250, 'COMM': '500', 'DEPTNO': 30}
{'EMPNO': 7566, 'ENAME': 'JONES', 'JOB': 'MANAGER', 'MGR': '7839', 'HIREDATE':
'2-Apr-06', 'SAL': 2975, 'COMM': '', 'DEPTNO': 20}
{'EMPNO': 7654, 'ENAME': 'MARTIN', 'JOB': 'SALESMAN', 'MGR': '7698', 'HIREDATE':
'28-Sep-06', 'SAL': 1250, 'COMM': '1400', 'DEPTNO': 30}
{'EMPNO': 7698, 'ENAME': 'BLAKE', 'JOB': 'MANAGER', 'MGR': '7839', 'HIREDATE':
'1-May-06', 'SAL': 2850, 'COMM': '', 'DEPTNO': 30}
{'EMPNO': 7782, 'ENAME': 'CLARK', 'JOB': 'MANAGER', 'MGR': '7839', 'HIREDATE':
'9-Jun-06', 'SAL': 2450, 'COMM': '', 'DEPTNO': 10}
{'EMPNO': 7788, 'ENAME': 'SCOTT', 'JOB': 'ANALYST', 'MGR': '7566', 'HIREDATE':
'9-Dec-07', 'SAL': 3000, 'COMM': '', 'DEPTNO': 20}
{'EMPNO': 7839, 'ENAME': 'KING', 'JOB': 'PRESIDENT', 'MGR': '', 'HIREDATE':
'17-Nov-06', 'SAL': 5000, 'COMM': '', 'DEPTNO': 10}
{'EMPNO': 7844, 'ENAME': 'TURNER', 'JOB': 'SALESMAN', 'MGR': '7698', 'HIREDATE':
'8-Sep-06', 'SAL': 1500, 'COMM': '0', 'DEPTNO': 30}
{'EMPNO': 7876, 'ENAME': 'ADAMS', 'JOB': 'CLERK', 'MGR': '7788', 'HIREDATE':
'12-Jan-08', 'SAL': 1100, 'COMM': '', 'DEPTNO': 20}
{'EMPNO': 7900, 'ENAME': 'JAMES', 'JOB': 'CLERK', 'MGR': '7698', 'HIREDATE':
'3-Dec-06', 'SAL': 950, 'COMM': '', 'DEPTNO': 30}
{'EMPNO': 7902, 'ENAME': 'FORD', 'JOB': 'ANALYST', 'MGR': '7566', 'HIREDATE':
'3-Dec-06', 'SAL': 3000, 'COMM': '', 'DEPTNO': 20}
{'EMPNO': 7934, 'ENAME': 'MILLER', 'JOB': 'CLERK', 'MGR': '7782', 'HIREDATE':
'23-Jan-07', 'SAL': 1300, 'COMM': '', 'DEPTNO': 10}
```

[43]:
```
new_cursor.execute('''SELECT * FROM dept''')
for dept_table in new_cursor:
    print(dept_table)
```

```
{'DEPTNO': 10, 'DNAME': 'ACCOUNTING', 'LOC': 'NEW YORK'}
{'DEPTNO': 20, 'DNAME': 'RESEARCH', 'LOC': 'DALLAS'}
{'DEPTNO': 30, 'DNAME': 'SALES', 'LOC': 'CHICAGO'}
{'DEPTNO': 40, 'DNAME': 'OPERATIONS', 'LOC': 'BOSTON'}
```

1.2.1 Removing unwanted characters from string

- For Example, removing char 'P' from the name

[44]:
```
new_cursor.execute('''select ename,
            replace(
            replace(
            replace(
            replace(
            replace(ename,'A',''),'E',''),'I',''),'O',''),'U','')
            as stripped1
            from emp;''')
for characters in new_cursor:
    print(characters)
```

```
{'ename': 'SMITH', 'stripped1': 'SMTH'}
{'ename': 'ALLEN', 'stripped1': 'LLN'}
{'ename': 'WARD', 'stripped1': 'WRD'}
{'ename': 'JONES', 'stripped1': 'JNS'}
{'ename': 'MARTIN', 'stripped1': 'MRTN'}
{'ename': 'BLAKE', 'stripped1': 'BLK'}
{'ename': 'CLARK', 'stripped1': 'CLRK'}
{'ename': 'SCOTT', 'stripped1': 'SCTT'}
{'ename': 'KING', 'stripped1': 'KNG'}
{'ename': 'TURNER', 'stripped1': 'TRNR'}
{'ename': 'ADAMS', 'stripped1': 'DMS'}
{'ename': 'JAMES', 'stripped1': 'JMS'}
{'ename': 'FORD', 'stripped1': 'FRD'}
{'ename': 'MILLER', 'stripped1': 'MLLR'}
```

Using the my_cursor object as we are connecting to sakila database to execute this query

[45]:
```
my_cursor.execute(''' SELECT first_name, REPLACE(first_name, 'P', '') as strp1␣
 ↪FROM actor LIMIT 1''')
for repc in my_cursor:
    print(repc)
```

```
{'first_name': 'PENELOPE', 'strp1': 'ENELOE'}
```

6.5 Separating Numeric and Character Data - Skip as it uses Translate function

6.6 Determining Whether a String Is Alphanumeric

[46]:
```
new_cursor.execute('''create view V as
    select ename as data
```

```
    from emp
    where deptno=10
    union all
    select concat(ename,', $',sal,'.00') as data
    from emp
    where deptno=20
    union all
    select concat(ename,deptno) as data
    from emp
    where deptno=30;''')
for V_view in new_cursor:
    print(V_view)
```

[47]:
```
new_cursor.execute('''select * from V''')
for V_view in new_cursor:
    print(V_view)
```

```
{'data': 'CLARK'}
{'data': 'KING'}
{'data': 'MILLER'}
{'data': 'SMITH, $800.00'}
{'data': 'JONES, $2975.00'}
{'data': 'SCOTT, $3000.00'}
{'data': 'ADAMS, $1100.00'}
{'data': 'FORD, $3000.00'}
{'data': 'ALLEN30'}
{'data': 'WARD30'}
{'data': 'MARTIN30'}
{'data': 'BLAKE30'}
{'data': 'TURNER30'}
{'data': 'JAMES30'}
```

[48]:
```
new_cursor.execute('''select data from V where data regexp '[^0-9a-zA-Z]' = 0;
  ↪''')
for V_view in new_cursor:
    print(V_view)
```

```
{'data': 'CLARK'}
{'data': 'KING'}
{'data': 'MILLER'}
{'data': 'ALLEN30'}
{'data': 'WARD30'}
{'data': 'MARTIN30'}
{'data': 'BLAKE30'}
{'data': 'TURNER30'}
{'data': 'JAMES30'}
```

6.7 Extracting Initials from a Name

Using the my_cursor object as we are connecting to sakila database for this query

[49]:
```
my_cursor.execute('''SELECT ((LENGTH('334 Munger (Monghyr) Lane')) -␣
 ↪LENGTH(REPLACE('334 Munger (Monghyr) Lane', '(', ''))) / LENGTH("(") AS cnt␣
 ↪''')
for cnt in my_cursor:
    print(cnt)
```

```
{'cnt': Decimal('1.0000')}
```

[50]:
```
my_cursor.execute('''
    SELECT * from actor
''')
for cnt in my_cursor:
    print(cnt)
```

```
{'actor_id': 1, 'first_name': 'PENELOPE', 'last_name': 'GUINESS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 2, 'first_name': 'NICK', 'last_name': 'WAHLBERG', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 3, 'first_name': 'ED', 'last_name': 'CHASE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 4, 'first_name': 'JENNIFER', 'last_name': 'DAVIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 5, 'first_name': 'JOHNNY', 'last_name': 'LOLLOBRIGIDA',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 6, 'first_name': 'BETTE', 'last_name': 'NICHOLSON', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 7, 'first_name': 'GRACE', 'last_name': 'MOSTEL', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 8, 'first_name': 'MATTHEW', 'last_name': 'JOHANSSON',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 9, 'first_name': 'JOE', 'last_name': 'SWANK', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 10, 'first_name': 'CHRISTIAN', 'last_name': 'GABLE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 11, 'first_name': 'ZERO', 'last_name': 'CAGE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 12, 'first_name': 'KARL', 'last_name': 'BERRY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 13, 'first_name': 'UMA', 'last_name': 'WOOD', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 14, 'first_name': 'VIVIEN', 'last_name': 'BERGEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 15, 'first_name': 'CUBA', 'last_name': 'OLIVIER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 16, 'first_name': 'FRED', 'last_name': 'COSTNER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
```

```
{'actor_id': 17, 'first_name': 'HELEN', 'last_name': 'VOIGHT', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 18, 'first_name': 'DAN', 'last_name': 'TORN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 19, 'first_name': 'BOB', 'last_name': 'FAWCETT', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 20, 'first_name': 'LUCILLE', 'last_name': 'TRACY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 21, 'first_name': 'KIRSTEN', 'last_name': 'PALTROW', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 22, 'first_name': 'ELVIS', 'last_name': 'MARX', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 23, 'first_name': 'SANDRA', 'last_name': 'KILMER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 24, 'first_name': 'CAMERON', 'last_name': 'STREEP', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 25, 'first_name': 'KEVIN', 'last_name': 'BLOOM', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 26, 'first_name': 'RIP', 'last_name': 'CRAWFORD', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 27, 'first_name': 'JULIA', 'last_name': 'MCQUEEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 28, 'first_name': 'WOODY', 'last_name': 'HOFFMAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 29, 'first_name': 'ALEC', 'last_name': 'WAYNE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 30, 'first_name': 'SANDRA', 'last_name': 'PECK', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 31, 'first_name': 'SISSY', 'last_name': 'SOBIESKI', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 32, 'first_name': 'TIM', 'last_name': 'HACKMAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 33, 'first_name': 'MILLA', 'last_name': 'PECK', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 34, 'first_name': 'AUDREY', 'last_name': 'OLIVIER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 35, 'first_name': 'JUDY', 'last_name': 'DEAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 36, 'first_name': 'BURT', 'last_name': 'DUKAKIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 37, 'first_name': 'VAL', 'last_name': 'BOLGER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 38, 'first_name': 'TOM', 'last_name': 'MCKELLEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 39, 'first_name': 'GOLDIE', 'last_name': 'BRODY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 40, 'first_name': 'JOHNNY', 'last_name': 'CAGE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
```

{'actor_id': 41, 'first_name': 'JODIE', 'last_name': 'DEGENERES', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 42, 'first_name': 'TOM', 'last_name': 'MIRANDA', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 43, 'first_name': 'KIRK', 'last_name': 'JOVOVICH', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 44, 'first_name': 'NICK', 'last_name': 'STALLONE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 45, 'first_name': 'REESE', 'last_name': 'KILMER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 46, 'first_name': 'PARKER', 'last_name': 'GOLDBERG', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 47, 'first_name': 'JULIA', 'last_name': 'BARRYMORE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 48, 'first_name': 'FRANCES', 'last_name': 'DAY-LEWIS',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 49, 'first_name': 'ANNE', 'last_name': 'CRONYN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 50, 'first_name': 'NATALIE', 'last_name': 'HOPKINS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 51, 'first_name': 'GARY', 'last_name': 'PHOENIX', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 52, 'first_name': 'CARMEN', 'last_name': 'HUNT', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 53, 'first_name': 'MENA', 'last_name': 'TEMPLE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 54, 'first_name': 'PENELOPE', 'last_name': 'PINKETT',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 55, 'first_name': 'FAY', 'last_name': 'KILMER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 56, 'first_name': 'DAN', 'last_name': 'HARRIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 57, 'first_name': 'JUDE', 'last_name': 'CRUISE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 58, 'first_name': 'CHRISTIAN', 'last_name': 'AKROYD',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 59, 'first_name': 'DUSTIN', 'last_name': 'TAUTOU', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 60, 'first_name': 'HENRY', 'last_name': 'BERRY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 61, 'first_name': 'CHRISTIAN', 'last_name': 'NEESON',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 62, 'first_name': 'JAYNE', 'last_name': 'NEESON', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 63, 'first_name': 'CAMERON', 'last_name': 'WRAY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 64, 'first_name': 'RAY', 'last_name': 'JOHANSSON', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}

{'actor_id': 65, 'first_name': 'ANGELA', 'last_name': 'HUDSON', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 66, 'first_name': 'MARY', 'last_name': 'TANDY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 67, 'first_name': 'JESSICA', 'last_name': 'BAILEY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 68, 'first_name': 'RIP', 'last_name': 'WINSLET', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 69, 'first_name': 'KENNETH', 'last_name': 'PALTROW', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 70, 'first_name': 'MICHELLE', 'last_name': 'MCCONAUGHEY',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 71, 'first_name': 'ADAM', 'last_name': 'GRANT', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 72, 'first_name': 'SEAN', 'last_name': 'WILLIAMS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 73, 'first_name': 'GARY', 'last_name': 'PENN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 74, 'first_name': 'MILLA', 'last_name': 'KEITEL', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 75, 'first_name': 'BURT', 'last_name': 'POSEY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 76, 'first_name': 'ANGELINA', 'last_name': 'ASTAIRE',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 77, 'first_name': 'CARY', 'last_name': 'MCCONAUGHEY',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 78, 'first_name': 'GROUCHO', 'last_name': 'SINATRA', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 79, 'first_name': 'MAE', 'last_name': 'HOFFMAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 80, 'first_name': 'RALPH', 'last_name': 'CRUZ', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 81, 'first_name': 'SCARLETT', 'last_name': 'DAMON', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 82, 'first_name': 'WOODY', 'last_name': 'JOLIE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 83, 'first_name': 'BEN', 'last_name': 'WILLIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 84, 'first_name': 'JAMES', 'last_name': 'PITT', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 85, 'first_name': 'MINNIE', 'last_name': 'ZELLWEGER',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 86, 'first_name': 'GREG', 'last_name': 'CHAPLIN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 87, 'first_name': 'SPENCER', 'last_name': 'PECK', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 88, 'first_name': 'KENNETH', 'last_name': 'PESCI', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}

{'actor_id': 89, 'first_name': 'CHARLIZE', 'last_name': 'DENCH', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 90, 'first_name': 'SEAN', 'last_name': 'GUINESS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 91, 'first_name': 'CHRISTOPHER', 'last_name': 'BERRY',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 92, 'first_name': 'KIRSTEN', 'last_name': 'AKROYD', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 93, 'first_name': 'ELLEN', 'last_name': 'PRESLEY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 94, 'first_name': 'KENNETH', 'last_name': 'TORN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 95, 'first_name': 'DARYL', 'last_name': 'WAHLBERG', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 96, 'first_name': 'GENE', 'last_name': 'WILLIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 97, 'first_name': 'MEG', 'last_name': 'HAWKE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 98, 'first_name': 'CHRIS', 'last_name': 'BRIDGES', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 99, 'first_name': 'JIM', 'last_name': 'MOSTEL', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 100, 'first_name': 'SPENCER', 'last_name': 'DEPP', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 101, 'first_name': 'SUSAN', 'last_name': 'DAVIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 102, 'first_name': 'WALTER', 'last_name': 'TORN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 103, 'first_name': 'MATTHEW', 'last_name': 'LEIGH', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 104, 'first_name': 'PENELOPE', 'last_name': 'CRONYN',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 105, 'first_name': 'SIDNEY', 'last_name': 'CROWE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 106, 'first_name': 'GROUCHO', 'last_name': 'DUNST', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 107, 'first_name': 'GINA', 'last_name': 'DEGENERES', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 108, 'first_name': 'WARREN', 'last_name': 'NOLTE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 109, 'first_name': 'SYLVESTER', 'last_name': 'DERN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 110, 'first_name': 'SUSAN', 'last_name': 'DAVIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 111, 'first_name': 'CAMERON', 'last_name': 'ZELLWEGER',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 112, 'first_name': 'RUSSELL', 'last_name': 'BACALL', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}

{'actor_id': 113, 'first_name': 'MORGAN', 'last_name': 'HOPKINS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 114, 'first_name': 'MORGAN', 'last_name': 'MCDORMAND',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 115, 'first_name': 'HARRISON', 'last_name': 'BALE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 116, 'first_name': 'DAN', 'last_name': 'STREEP', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 117, 'first_name': 'RENEE', 'last_name': 'TRACY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 118, 'first_name': 'CUBA', 'last_name': 'ALLEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 119, 'first_name': 'WARREN', 'last_name': 'JACKMAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 120, 'first_name': 'PENELOPE', 'last_name': 'MONROE',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 121, 'first_name': 'LIZA', 'last_name': 'BERGMAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 122, 'first_name': 'SALMA', 'last_name': 'NOLTE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 123, 'first_name': 'JULIANNE', 'last_name': 'DENCH', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 124, 'first_name': 'SCARLETT', 'last_name': 'BENING',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 125, 'first_name': 'ALBERT', 'last_name': 'NOLTE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 126, 'first_name': 'FRANCES', 'last_name': 'TOMEI', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 127, 'first_name': 'KEVIN', 'last_name': 'GARLAND', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 128, 'first_name': 'CATE', 'last_name': 'MCQUEEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 129, 'first_name': 'DARYL', 'last_name': 'CRAWFORD', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 130, 'first_name': 'GRETA', 'last_name': 'KEITEL', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 131, 'first_name': 'JANE', 'last_name': 'JACKMAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 132, 'first_name': 'ADAM', 'last_name': 'HOPPER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 133, 'first_name': 'RICHARD', 'last_name': 'PENN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 134, 'first_name': 'GENE', 'last_name': 'HOPKINS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 135, 'first_name': 'RITA', 'last_name': 'REYNOLDS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 136, 'first_name': 'ED', 'last_name': 'MANSFIELD', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}

{'actor_id': 137, 'first_name': 'MORGAN', 'last_name': 'WILLIAMS',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 138, 'first_name': 'LUCILLE', 'last_name': 'DEE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 139, 'first_name': 'EWAN', 'last_name': 'GOODING', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 140, 'first_name': 'WHOOPI', 'last_name': 'HURT', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 141, 'first_name': 'CATE', 'last_name': 'HARRIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 142, 'first_name': 'JADA', 'last_name': 'RYDER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 143, 'first_name': 'RIVER', 'last_name': 'DEAN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 144, 'first_name': 'ANGELA', 'last_name': 'WITHERSPOON',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 145, 'first_name': 'KIM', 'last_name': 'ALLEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 146, 'first_name': 'ALBERT', 'last_name': 'JOHANSSON',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 147, 'first_name': 'FAY', 'last_name': 'WINSLET', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 148, 'first_name': 'EMILY', 'last_name': 'DEE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 149, 'first_name': 'RUSSELL', 'last_name': 'TEMPLE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 150, 'first_name': 'JAYNE', 'last_name': 'NOLTE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 151, 'first_name': 'GEOFFREY', 'last_name': 'HESTON',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 152, 'first_name': 'BEN', 'last_name': 'HARRIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 153, 'first_name': 'MINNIE', 'last_name': 'KILMER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 154, 'first_name': 'MERYL', 'last_name': 'GIBSON', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 155, 'first_name': 'IAN', 'last_name': 'TANDY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 156, 'first_name': 'FAY', 'last_name': 'WOOD', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 157, 'first_name': 'GRETA', 'last_name': 'MALDEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 158, 'first_name': 'VIVIEN', 'last_name': 'BASINGER',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 159, 'first_name': 'LAURA', 'last_name': 'BRODY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 160, 'first_name': 'CHRIS', 'last_name': 'DEPP', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}

{'actor_id': 161, 'first_name': 'HARVEY', 'last_name': 'HOPE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 162, 'first_name': 'OPRAH', 'last_name': 'KILMER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 163, 'first_name': 'CHRISTOPHER', 'last_name': 'WEST',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 164, 'first_name': 'HUMPHREY', 'last_name': 'WILLIS',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 165, 'first_name': 'AL', 'last_name': 'GARLAND', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 166, 'first_name': 'NICK', 'last_name': 'DEGENERES', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 167, 'first_name': 'LAURENCE', 'last_name': 'BULLOCK',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 168, 'first_name': 'WILL', 'last_name': 'WILSON', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 169, 'first_name': 'KENNETH', 'last_name': 'HOFFMAN',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 170, 'first_name': 'MENA', 'last_name': 'HOPPER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 171, 'first_name': 'OLYMPIA', 'last_name': 'PFEIFFER',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 172, 'first_name': 'GROUCHO', 'last_name': 'WILLIAMS',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 173, 'first_name': 'ALAN', 'last_name': 'DREYFUSS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 174, 'first_name': 'MICHAEL', 'last_name': 'BENING', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 175, 'first_name': 'WILLIAM', 'last_name': 'HACKMAN',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 176, 'first_name': 'JON', 'last_name': 'CHASE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 177, 'first_name': 'GENE', 'last_name': 'MCKELLEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 178, 'first_name': 'LISA', 'last_name': 'MONROE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 179, 'first_name': 'ED', 'last_name': 'GUINESS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 180, 'first_name': 'JEFF', 'last_name': 'SILVERSTONE',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 181, 'first_name': 'MATTHEW', 'last_name': 'CARREY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 182, 'first_name': 'DEBBIE', 'last_name': 'AKROYD', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 183, 'first_name': 'RUSSELL', 'last_name': 'CLOSE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 184, 'first_name': 'HUMPHREY', 'last_name': 'GARLAND',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}

```
{'actor_id': 185, 'first_name': 'MICHAEL', 'last_name': 'BOLGER', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 186, 'first_name': 'JULIA', 'last_name': 'ZELLWEGER',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 187, 'first_name': 'RENEE', 'last_name': 'BALL', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 188, 'first_name': 'ROCK', 'last_name': 'DUKAKIS', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 189, 'first_name': 'CUBA', 'last_name': 'BIRCH', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 190, 'first_name': 'AUDREY', 'last_name': 'BAILEY', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 191, 'first_name': 'GREGORY', 'last_name': 'GOODING',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 192, 'first_name': 'JOHN', 'last_name': 'SUVARI', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 193, 'first_name': 'BURT', 'last_name': 'TEMPLE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 194, 'first_name': 'MERYL', 'last_name': 'ALLEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 195, 'first_name': 'JAYNE', 'last_name': 'SILVERSTONE',
'last_update': datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 196, 'first_name': 'BELA', 'last_name': 'WALKEN', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 197, 'first_name': 'REESE', 'last_name': 'WEST', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 198, 'first_name': 'MARY', 'last_name': 'KEITEL', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 199, 'first_name': 'JULIA', 'last_name': 'FAWCETT', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
{'actor_id': 200, 'first_name': 'THORA', 'last_name': 'TEMPLE', 'last_update':
datetime.datetime(2006, 2, 15, 4, 34, 33)}
```

[51]:
```
my_cursor.execute('''
    SELECT first_name, last_name, CONCAT(SUBSTRING(SUBSTRING_INDEX(first_name,␣
 ↪' ', 1), 1, 1), '.', SUBSTRING(SUBSTRING_INDEX(last_name, ' ', -1), 1, 1))␣
 ↪AS initials
    FROM actor;
''')
for cnt in my_cursor:
    print(cnt)
```

```
{'first_name': 'PENELOPE', 'last_name': 'GUINESS', 'initials': 'P.G'}
{'first_name': 'NICK', 'last_name': 'WAHLBERG', 'initials': 'N.W'}
{'first_name': 'ED', 'last_name': 'CHASE', 'initials': 'E.C'}
{'first_name': 'JENNIFER', 'last_name': 'DAVIS', 'initials': 'J.D'}
{'first_name': 'JOHNNY', 'last_name': 'LOLLOBRIGIDA', 'initials': 'J.L'}
{'first_name': 'BETTE', 'last_name': 'NICHOLSON', 'initials': 'B.N'}
```

{'first_name': 'GRACE', 'last_name': 'MOSTEL', 'initials': 'G.M'}
{'first_name': 'MATTHEW', 'last_name': 'JOHANSSON', 'initials': 'M.J'}
{'first_name': 'JOE', 'last_name': 'SWANK', 'initials': 'J.S'}
{'first_name': 'CHRISTIAN', 'last_name': 'GABLE', 'initials': 'C.G'}
{'first_name': 'ZERO', 'last_name': 'CAGE', 'initials': 'Z.C'}
{'first_name': 'KARL', 'last_name': 'BERRY', 'initials': 'K.B'}
{'first_name': 'UMA', 'last_name': 'WOOD', 'initials': 'U.W'}
{'first_name': 'VIVIEN', 'last_name': 'BERGEN', 'initials': 'V.B'}
{'first_name': 'CUBA', 'last_name': 'OLIVIER', 'initials': 'C.O'}
{'first_name': 'FRED', 'last_name': 'COSTNER', 'initials': 'F.C'}
{'first_name': 'HELEN', 'last_name': 'VOIGHT', 'initials': 'H.V'}
{'first_name': 'DAN', 'last_name': 'TORN', 'initials': 'D.T'}
{'first_name': 'BOB', 'last_name': 'FAWCETT', 'initials': 'B.F'}
{'first_name': 'LUCILLE', 'last_name': 'TRACY', 'initials': 'L.T'}
{'first_name': 'KIRSTEN', 'last_name': 'PALTROW', 'initials': 'K.P'}
{'first_name': 'ELVIS', 'last_name': 'MARX', 'initials': 'E.M'}
{'first_name': 'SANDRA', 'last_name': 'KILMER', 'initials': 'S.K'}
{'first_name': 'CAMERON', 'last_name': 'STREEP', 'initials': 'C.S'}
{'first_name': 'KEVIN', 'last_name': 'BLOOM', 'initials': 'K.B'}
{'first_name': 'RIP', 'last_name': 'CRAWFORD', 'initials': 'R.C'}
{'first_name': 'JULIA', 'last_name': 'MCQUEEN', 'initials': 'J.M'}
{'first_name': 'WOODY', 'last_name': 'HOFFMAN', 'initials': 'W.H'}
{'first_name': 'ALEC', 'last_name': 'WAYNE', 'initials': 'A.W'}
{'first_name': 'SANDRA', 'last_name': 'PECK', 'initials': 'S.P'}
{'first_name': 'SISSY', 'last_name': 'SOBIESKI', 'initials': 'S.S'}
{'first_name': 'TIM', 'last_name': 'HACKMAN', 'initials': 'T.H'}
{'first_name': 'MILLA', 'last_name': 'PECK', 'initials': 'M.P'}
{'first_name': 'AUDREY', 'last_name': 'OLIVIER', 'initials': 'A.O'}
{'first_name': 'JUDY', 'last_name': 'DEAN', 'initials': 'J.D'}
{'first_name': 'BURT', 'last_name': 'DUKAKIS', 'initials': 'B.D'}
{'first_name': 'VAL', 'last_name': 'BOLGER', 'initials': 'V.B'}
{'first_name': 'TOM', 'last_name': 'MCKELLEN', 'initials': 'T.M'}
{'first_name': 'GOLDIE', 'last_name': 'BRODY', 'initials': 'G.B'}
{'first_name': 'JOHNNY', 'last_name': 'CAGE', 'initials': 'J.C'}
{'first_name': 'JODIE', 'last_name': 'DEGENERES', 'initials': 'J.D'}
{'first_name': 'TOM', 'last_name': 'MIRANDA', 'initials': 'T.M'}
{'first_name': 'KIRK', 'last_name': 'JOVOVICH', 'initials': 'K.J'}
{'first_name': 'NICK', 'last_name': 'STALLONE', 'initials': 'N.S'}
{'first_name': 'REESE', 'last_name': 'KILMER', 'initials': 'R.K'}
{'first_name': 'PARKER', 'last_name': 'GOLDBERG', 'initials': 'P.G'}
{'first_name': 'JULIA', 'last_name': 'BARRYMORE', 'initials': 'J.B'}
{'first_name': 'FRANCES', 'last_name': 'DAY-LEWIS', 'initials': 'F.D'}
{'first_name': 'ANNE', 'last_name': 'CRONYN', 'initials': 'A.C'}
{'first_name': 'NATALIE', 'last_name': 'HOPKINS', 'initials': 'N.H'}
{'first_name': 'GARY', 'last_name': 'PHOENIX', 'initials': 'G.P'}
{'first_name': 'CARMEN', 'last_name': 'HUNT', 'initials': 'C.H'}
{'first_name': 'MENA', 'last_name': 'TEMPLE', 'initials': 'M.T'}
{'first_name': 'PENELOPE', 'last_name': 'PINKETT', 'initials': 'P.P'}

{'first_name': 'FAY', 'last_name': 'KILMER', 'initials': 'F.K'}
{'first_name': 'DAN', 'last_name': 'HARRIS', 'initials': 'D.H'}
{'first_name': 'JUDE', 'last_name': 'CRUISE', 'initials': 'J.C'}
{'first_name': 'CHRISTIAN', 'last_name': 'AKROYD', 'initials': 'C.A'}
{'first_name': 'DUSTIN', 'last_name': 'TAUTOU', 'initials': 'D.T'}
{'first_name': 'HENRY', 'last_name': 'BERRY', 'initials': 'H.B'}
{'first_name': 'CHRISTIAN', 'last_name': 'NEESON', 'initials': 'C.N'}
{'first_name': 'JAYNE', 'last_name': 'NEESON', 'initials': 'J.N'}
{'first_name': 'CAMERON', 'last_name': 'WRAY', 'initials': 'C.W'}
{'first_name': 'RAY', 'last_name': 'JOHANSSON', 'initials': 'R.J'}
{'first_name': 'ANGELA', 'last_name': 'HUDSON', 'initials': 'A.H'}
{'first_name': 'MARY', 'last_name': 'TANDY', 'initials': 'M.T'}
{'first_name': 'JESSICA', 'last_name': 'BAILEY', 'initials': 'J.B'}
{'first_name': 'RIP', 'last_name': 'WINSLET', 'initials': 'R.W'}
{'first_name': 'KENNETH', 'last_name': 'PALTROW', 'initials': 'K.P'}
{'first_name': 'MICHELLE', 'last_name': 'MCCONAUGHEY', 'initials': 'M.M'}
{'first_name': 'ADAM', 'last_name': 'GRANT', 'initials': 'A.G'}
{'first_name': 'SEAN', 'last_name': 'WILLIAMS', 'initials': 'S.W'}
{'first_name': 'GARY', 'last_name': 'PENN', 'initials': 'G.P'}
{'first_name': 'MILLA', 'last_name': 'KEITEL', 'initials': 'M.K'}
{'first_name': 'BURT', 'last_name': 'POSEY', 'initials': 'B.P'}
{'first_name': 'ANGELINA', 'last_name': 'ASTAIRE', 'initials': 'A.A'}
{'first_name': 'CARY', 'last_name': 'MCCONAUGHEY', 'initials': 'C.M'}
{'first_name': 'GROUCHO', 'last_name': 'SINATRA', 'initials': 'G.S'}
{'first_name': 'MAE', 'last_name': 'HOFFMAN', 'initials': 'M.H'}
{'first_name': 'RALPH', 'last_name': 'CRUZ', 'initials': 'R.C'}
{'first_name': 'SCARLETT', 'last_name': 'DAMON', 'initials': 'S.D'}
{'first_name': 'WOODY', 'last_name': 'JOLIE', 'initials': 'W.J'}
{'first_name': 'BEN', 'last_name': 'WILLIS', 'initials': 'B.W'}
{'first_name': 'JAMES', 'last_name': 'PITT', 'initials': 'J.P'}
{'first_name': 'MINNIE', 'last_name': 'ZELLWEGER', 'initials': 'M.Z'}
{'first_name': 'GREG', 'last_name': 'CHAPLIN', 'initials': 'G.C'}
{'first_name': 'SPENCER', 'last_name': 'PECK', 'initials': 'S.P'}
{'first_name': 'KENNETH', 'last_name': 'PESCI', 'initials': 'K.P'}
{'first_name': 'CHARLIZE', 'last_name': 'DENCH', 'initials': 'C.D'}
{'first_name': 'SEAN', 'last_name': 'GUINESS', 'initials': 'S.G'}
{'first_name': 'CHRISTOPHER', 'last_name': 'BERRY', 'initials': 'C.B'}
{'first_name': 'KIRSTEN', 'last_name': 'AKROYD', 'initials': 'K.A'}
{'first_name': 'ELLEN', 'last_name': 'PRESLEY', 'initials': 'E.P'}
{'first_name': 'KENNETH', 'last_name': 'TORN', 'initials': 'K.T'}
{'first_name': 'DARYL', 'last_name': 'WAHLBERG', 'initials': 'D.W'}
{'first_name': 'GENE', 'last_name': 'WILLIS', 'initials': 'G.W'}
{'first_name': 'MEG', 'last_name': 'HAWKE', 'initials': 'M.H'}
{'first_name': 'CHRIS', 'last_name': 'BRIDGES', 'initials': 'C.B'}
{'first_name': 'JIM', 'last_name': 'MOSTEL', 'initials': 'J.M'}
{'first_name': 'SPENCER', 'last_name': 'DEPP', 'initials': 'S.D'}
{'first_name': 'SUSAN', 'last_name': 'DAVIS', 'initials': 'S.D'}
{'first_name': 'WALTER', 'last_name': 'TORN', 'initials': 'W.T'}

{'first_name': 'MATTHEW', 'last_name': 'LEIGH', 'initials': 'M.L'}
{'first_name': 'PENELOPE', 'last_name': 'CRONYN', 'initials': 'P.C'}
{'first_name': 'SIDNEY', 'last_name': 'CROWE', 'initials': 'S.C'}
{'first_name': 'GROUCHO', 'last_name': 'DUNST', 'initials': 'G.D'}
{'first_name': 'GINA', 'last_name': 'DEGENERES', 'initials': 'G.D'}
{'first_name': 'WARREN', 'last_name': 'NOLTE', 'initials': 'W.N'}
{'first_name': 'SYLVESTER', 'last_name': 'DERN', 'initials': 'S.D'}
{'first_name': 'SUSAN', 'last_name': 'DAVIS', 'initials': 'S.D'}
{'first_name': 'CAMERON', 'last_name': 'ZELLWEGER', 'initials': 'C.Z'}
{'first_name': 'RUSSELL', 'last_name': 'BACALL', 'initials': 'R.B'}
{'first_name': 'MORGAN', 'last_name': 'HOPKINS', 'initials': 'M.H'}
{'first_name': 'MORGAN', 'last_name': 'MCDORMAND', 'initials': 'M.M'}
{'first_name': 'HARRISON', 'last_name': 'BALE', 'initials': 'H.B'}
{'first_name': 'DAN', 'last_name': 'STREEP', 'initials': 'D.S'}
{'first_name': 'RENEE', 'last_name': 'TRACY', 'initials': 'R.T'}
{'first_name': 'CUBA', 'last_name': 'ALLEN', 'initials': 'C.A'}
{'first_name': 'WARREN', 'last_name': 'JACKMAN', 'initials': 'W.J'}
{'first_name': 'PENELOPE', 'last_name': 'MONROE', 'initials': 'P.M'}
{'first_name': 'LIZA', 'last_name': 'BERGMAN', 'initials': 'L.B'}
{'first_name': 'SALMA', 'last_name': 'NOLTE', 'initials': 'S.N'}
{'first_name': 'JULIANNE', 'last_name': 'DENCH', 'initials': 'J.D'}
{'first_name': 'SCARLETT', 'last_name': 'BENING', 'initials': 'S.B'}
{'first_name': 'ALBERT', 'last_name': 'NOLTE', 'initials': 'A.N'}
{'first_name': 'FRANCES', 'last_name': 'TOMEI', 'initials': 'F.T'}
{'first_name': 'KEVIN', 'last_name': 'GARLAND', 'initials': 'K.G'}
{'first_name': 'CATE', 'last_name': 'MCQUEEN', 'initials': 'C.M'}
{'first_name': 'DARYL', 'last_name': 'CRAWFORD', 'initials': 'D.C'}
{'first_name': 'GRETA', 'last_name': 'KEITEL', 'initials': 'G.K'}
{'first_name': 'JANE', 'last_name': 'JACKMAN', 'initials': 'J.J'}
{'first_name': 'ADAM', 'last_name': 'HOPPER', 'initials': 'A.H'}
{'first_name': 'RICHARD', 'last_name': 'PENN', 'initials': 'R.P'}
{'first_name': 'GENE', 'last_name': 'HOPKINS', 'initials': 'G.H'}
{'first_name': 'RITA', 'last_name': 'REYNOLDS', 'initials': 'R.R'}
{'first_name': 'ED', 'last_name': 'MANSFIELD', 'initials': 'E.M'}
{'first_name': 'MORGAN', 'last_name': 'WILLIAMS', 'initials': 'M.W'}
{'first_name': 'LUCILLE', 'last_name': 'DEE', 'initials': 'L.D'}
{'first_name': 'EWAN', 'last_name': 'GOODING', 'initials': 'E.G'}
{'first_name': 'WHOOPI', 'last_name': 'HURT', 'initials': 'W.H'}
{'first_name': 'CATE', 'last_name': 'HARRIS', 'initials': 'C.H'}
{'first_name': 'JADA', 'last_name': 'RYDER', 'initials': 'J.R'}
{'first_name': 'RIVER', 'last_name': 'DEAN', 'initials': 'R.D'}
{'first_name': 'ANGELA', 'last_name': 'WITHERSPOON', 'initials': 'A.W'}
{'first_name': 'KIM', 'last_name': 'ALLEN', 'initials': 'K.A'}
{'first_name': 'ALBERT', 'last_name': 'JOHANSSON', 'initials': 'A.J'}
{'first_name': 'FAY', 'last_name': 'WINSLET', 'initials': 'F.W'}
{'first_name': 'EMILY', 'last_name': 'DEE', 'initials': 'E.D'}
{'first_name': 'RUSSELL', 'last_name': 'TEMPLE', 'initials': 'R.T'}
{'first_name': 'JAYNE', 'last_name': 'NOLTE', 'initials': 'J.N'}

{'first_name': 'GEOFFREY', 'last_name': 'HESTON', 'initials': 'G.H'}
{'first_name': 'BEN', 'last_name': 'HARRIS', 'initials': 'B.H'}
{'first_name': 'MINNIE', 'last_name': 'KILMER', 'initials': 'M.K'}
{'first_name': 'MERYL', 'last_name': 'GIBSON', 'initials': 'M.G'}
{'first_name': 'IAN', 'last_name': 'TANDY', 'initials': 'I.T'}
{'first_name': 'FAY', 'last_name': 'WOOD', 'initials': 'F.W'}
{'first_name': 'GRETA', 'last_name': 'MALDEN', 'initials': 'G.M'}
{'first_name': 'VIVIEN', 'last_name': 'BASINGER', 'initials': 'V.B'}
{'first_name': 'LAURA', 'last_name': 'BRODY', 'initials': 'L.B'}
{'first_name': 'CHRIS', 'last_name': 'DEPP', 'initials': 'C.D'}
{'first_name': 'HARVEY', 'last_name': 'HOPE', 'initials': 'H.H'}
{'first_name': 'OPRAH', 'last_name': 'KILMER', 'initials': 'O.K'}
{'first_name': 'CHRISTOPHER', 'last_name': 'WEST', 'initials': 'C.W'}
{'first_name': 'HUMPHREY', 'last_name': 'WILLIS', 'initials': 'H.W'}
{'first_name': 'AL', 'last_name': 'GARLAND', 'initials': 'A.G'}
{'first_name': 'NICK', 'last_name': 'DEGENERES', 'initials': 'N.D'}
{'first_name': 'LAURENCE', 'last_name': 'BULLOCK', 'initials': 'L.B'}
{'first_name': 'WILL', 'last_name': 'WILSON', 'initials': 'W.W'}
{'first_name': 'KENNETH', 'last_name': 'HOFFMAN', 'initials': 'K.H'}
{'first_name': 'MENA', 'last_name': 'HOPPER', 'initials': 'M.H'}
{'first_name': 'OLYMPIA', 'last_name': 'PFEIFFER', 'initials': 'O.P'}
{'first_name': 'GROUCHO', 'last_name': 'WILLIAMS', 'initials': 'G.W'}
{'first_name': 'ALAN', 'last_name': 'DREYFUSS', 'initials': 'A.D'}
{'first_name': 'MICHAEL', 'last_name': 'BENING', 'initials': 'M.B'}
{'first_name': 'WILLIAM', 'last_name': 'HACKMAN', 'initials': 'W.H'}
{'first_name': 'JON', 'last_name': 'CHASE', 'initials': 'J.C'}
{'first_name': 'GENE', 'last_name': 'MCKELLEN', 'initials': 'G.M'}
{'first_name': 'LISA', 'last_name': 'MONROE', 'initials': 'L.M'}
{'first_name': 'ED', 'last_name': 'GUINESS', 'initials': 'E.G'}
{'first_name': 'JEFF', 'last_name': 'SILVERSTONE', 'initials': 'J.S'}
{'first_name': 'MATTHEW', 'last_name': 'CARREY', 'initials': 'M.C'}
{'first_name': 'DEBBIE', 'last_name': 'AKROYD', 'initials': 'D.A'}
{'first_name': 'RUSSELL', 'last_name': 'CLOSE', 'initials': 'R.C'}
{'first_name': 'HUMPHREY', 'last_name': 'GARLAND', 'initials': 'H.G'}
{'first_name': 'MICHAEL', 'last_name': 'BOLGER', 'initials': 'M.B'}
{'first_name': 'JULIA', 'last_name': 'ZELLWEGER', 'initials': 'J.Z'}
{'first_name': 'RENEE', 'last_name': 'BALL', 'initials': 'R.B'}
{'first_name': 'ROCK', 'last_name': 'DUKAKIS', 'initials': 'R.D'}
{'first_name': 'CUBA', 'last_name': 'BIRCH', 'initials': 'C.B'}
{'first_name': 'AUDREY', 'last_name': 'BAILEY', 'initials': 'A.B'}
{'first_name': 'GREGORY', 'last_name': 'GOODING', 'initials': 'G.G'}
{'first_name': 'JOHN', 'last_name': 'SUVARI', 'initials': 'J.S'}
{'first_name': 'BURT', 'last_name': 'TEMPLE', 'initials': 'B.T'}
{'first_name': 'MERYL', 'last_name': 'ALLEN', 'initials': 'M.A'}
{'first_name': 'JAYNE', 'last_name': 'SILVERSTONE', 'initials': 'J.S'}
{'first_name': 'BELA', 'last_name': 'WALKEN', 'initials': 'B.W'}
{'first_name': 'REESE', 'last_name': 'WEST', 'initials': 'R.W'}
{'first_name': 'MARY', 'last_name': 'KEITEL', 'initials': 'M.K'}

```
{'first_name': 'JULIA', 'last_name': 'FAWCETT', 'initials': 'J.F'}
{'first_name': 'THORA', 'last_name': 'TEMPLE', 'initials': 'T.T'}
```

6.8 Ordering by Parts of a String

[52]:
```
## ordering by last two characters
new_cursor.execute('''select ename from emp order by␣
 ↪substr(ename,length(ename)-1,2);''')
for sub in new_cursor:
    print(sub)
```

```
{'ename': 'ALLEN'}
{'ename': 'TURNER'}
{'ename': 'MILLER'}
{'ename': 'JONES'}
{'ename': 'JAMES'}
{'ename': 'MARTIN'}
{'ename': 'BLAKE'}
{'ename': 'ADAMS'}
{'ename': 'KING'}
{'ename': 'WARD'}
{'ename': 'FORD'}
{'ename': 'CLARK'}
{'ename': 'SMITH'}
{'ename': 'SCOTT'}
```

[53]:
```
my_cursor.execute(''' SELECT title FROM film ORDER BY SUBSTR(title,␣
 ↪LENGTH(title) - 1) LIMIT 5; ''')
conc_res = my_cursor.fetchall()
print(conc_res)
```

```
[{'title': 'PACIFIC AMISTAD'}, {'title': 'FROST HEAD'}, {'title': 'CATCH
AMISTAD'}, {'title': 'DRAGON SQUAD'}, {'title': 'COMA HEAD'}]
```

6.9 Ordering by a Number in a String - Cannot execute as MySQL does not provide the TRANSLATE function

6.10 Creating a Delimited List from Table Rows

[54]:
```
new_cursor.execute('''select deptno, group_concat(ename order by empno␣
 ↪separator ',') as emps from emp group by deptno;''')
for delimit in new_cursor:
    print(delimit)
```

```
{'deptno': 10, 'emps': 'CLARK,KING,MILLER'}
{'deptno': 20, 'emps': 'SMITH,JONES,SCOTT,ADAMS,FORD'}
{'deptno': 30, 'emps': 'ALLEN,WARD,MARTIN,BLAKE,TURNER,JAMES'}
```

6.11 Converting Delimited Data into a Multivalued IN-List

[55]:
```
## here, the IN LIST ( '7654,7698,7782,7788' ) is considered to be a string␣
 ↪rather than a comma delimited list
new_cursor.execute('''select empno, ename,sal,deptno
    from emp
    where empno in ( 7654,7698,7782,7788 );''')
for delimit in new_cursor:
    print(delimit)
```

```
{'empno': 7654, 'ename': 'MARTIN', 'sal': 1250, 'deptno': 30}
{'empno': 7698, 'ename': 'BLAKE', 'sal': 2850, 'deptno': 30}
{'empno': 7782, 'ename': 'CLARK', 'sal': 2450, 'deptno': 10}
{'empno': 7788, 'ename': 'SCOTT', 'sal': 3000, 'deptno': 20}
```

[56]:
```
new_cursor.execute('''select ename,sal,deptno
    from emp
    where empno= 7654;''')
for delimit in new_cursor:
    print(delimit)
```

```
{'ename': 'MARTIN', 'sal': 1250, 'deptno': 30}
```

[57]:
```
new_cursor.execute('''CREATE TEMPORARY TABLE temp_numbers (id INT);''')
for temp in new_cursor:
    print(temp)
```

[58]:
```
new_cursor.execute('''INSERT INTO temp_numbers (id) VALUES
(1), (2), (3), (4), (5), (6), (7), (8), (9), (10),
(11), (12), (13), (14), (15), (16), (17), (18), (19), (20);''')
for temp in new_cursor:
    print(temp)
```

[59]:
```
new_cursor.execute('''select empno, ename, sal, deptno
    from emp
    where empno in
    ( select substring_index(
     substring_index(list.vals,',',iter.id),',',-1) empno
    from temp_numbers as iter,
    (select '7654,7698,7782,7788' as vals
    from (select 1) as dummy_table
    )list
    where iter.id <=
    (length(list.vals)-length(replace(list.vals,',','')))+1
    );''')
for delimit in new_cursor:
    print(delimit)
```

```
{'empno': 7654, 'ename': 'MARTIN', 'sal': 1250, 'deptno': 30}
{'empno': 7698, 'ename': 'BLAKE', 'sal': 2850, 'deptno': 30}
```

```
{'empno': 7782, 'ename': 'CLARK', 'sal': 2450, 'deptno': 10}
{'empno': 7788, 'ename': 'SCOTT', 'sal': 3000, 'deptno': 20}
```

6.12 Alphabetizing a String

[60]:
```
new_cursor.execute('''select ename, group_concat(c order by c separator '') AS␣
 ↪sorted_ename
    from (
    select ename, substr(a.ename,iter.id,1) c
    from emp a, temp_numbers iter
    WHERE iter.id <= LENGTH(a.ename)
    ) x
    GROUP BY ename;''')
for alpha in new_cursor:
    print(alpha)
```

```
{'ename': 'ADAMS', 'sorted_ename': 'AADMS'}
{'ename': 'ALLEN', 'sorted_ename': 'AELLN'}
{'ename': 'BLAKE', 'sorted_ename': 'ABEKL'}
{'ename': 'CLARK', 'sorted_ename': 'ACKLR'}
{'ename': 'FORD', 'sorted_ename': 'DFOR'}
{'ename': 'JAMES', 'sorted_ename': 'AEJMS'}
{'ename': 'JONES', 'sorted_ename': 'EJNOS'}
{'ename': 'KING', 'sorted_ename': 'GIKN'}
{'ename': 'MARTIN', 'sorted_ename': 'AIMNRT'}
{'ename': 'MILLER', 'sorted_ename': 'EILLMR'}
{'ename': 'SCOTT', 'sorted_ename': 'COSTT'}
{'ename': 'SMITH', 'sorted_ename': 'HIMST'}
{'ename': 'TURNER', 'sorted_ename': 'ENRRTU'}
{'ename': 'WARD', 'sorted_ename': 'ADRW'}
```

6.13 Identifying Strings That Can Be Treated as Numbers

[61]:
```
new_cursor.execute('''create view P as
    select concat(
    substr(ename,1,2),
    replace(cast(deptno as char(4)),' ',''),
    substr(ename,3,2)
    ) as mixed
    from emp
    where deptno = 10
    union all
    select replace(cast(empno as char(4)), ' ', '')
    from emp where deptno = 20
    union all
    select ename from emp where deptno = 30;''')
for View_P in new_cursor:
    print(View_P)
```

[62]:
```
new_cursor.execute('''select * from P''')
for View_P in new_cursor:
    print(View_P)
```

```
{'mixed': 'CL10AR'}
{'mixed': 'KI10NG'}
{'mixed': 'MI10LL'}
{'mixed': '7369'}
{'mixed': '7566'}
{'mixed': '7788'}
{'mixed': '7876'}
{'mixed': '7902'}
{'mixed': 'ALLEN'}
{'mixed': 'WARD'}
{'mixed': 'MARTIN'}
{'mixed': 'BLAKE'}
{'mixed': 'TURNER'}
{'mixed': 'JAMES'}
```

[63]:
```
new_cursor.execute('''select cast(group_concat(c order by pos separator '') as␣
 ↪unsigned)
        as MIXED1
        from (
        select P.mixed, iter.id AS pos, substr(P.mixed,iter.id,1) as c from P,
        temp_numbers as iter where iter.id <= length(P.mixed)
        and ascii(substr(P.mixed,iter.id,1)) between 48 and 57)y

        group by mixed
        order by 1;''')
for str in new_cursor:
    print(str)
```

```
{'MIXED1': 10}
{'MIXED1': 10}
{'MIXED1': 10}
{'MIXED1': 7369}
{'MIXED1': 7566}
{'MIXED1': 7788}
{'MIXED1': 7876}
{'MIXED1': 7902}
```

6.14 Extracting the nth Delimited Substring

[64]:
```
new_cursor.execute('''CREATE TABLE permanent_numbers (id INT PRIMARY KEY);''')
for str in new_cursor:
    print(str)
```

[65]:
```
new_cursor.execute('''INSERT INTO permanent_numbers VALUES (1), (2), (3),(4),␣
 ↪(5), (6), (7), (8), (9), (10),
(11), (12), (13), (14), (15), (16), (17), (18), (19), (20);
''')
for temp in new_cursor:
    print(temp)
```

[66]:
```
new_cursor.execute('''CREATE VIEW C AS
    SELECT 'mo,larry,curly' AS name
    FROM permanent_numbers
    UNION ALL
    SELECT 'tina,gina,jaunita,regina,leena' AS name
    FROM permanent_numbers;''')
for View_C in new_cursor:
    print(View_C)
```

[67]:
```
new_cursor.execute('''select distinct  * from C;''')
for View_C in new_cursor:
    print(View_C)
```

```
{'name': 'mo,larry,curly'}
{'name': 'tina,gina,jaunita,regina,leena'}
```

[ ]:

[68]:
```
new_cursor.execute('''Select distinct name
        from (
        select iter.id AS pos,
        substring_index(
        substring_index(src.name,',',iter.id),',',-1) AS name
        from C src,
        temp_numbers iter
        where iter.id <=
        length(src.name)-length(replace(src.name,',',''))
        ) x
        where pos = 2 LIMIT 2;''')
for View_C in new_cursor:
    print(View_C)
```

```
{'name': 'larry'}
{'name': 'gina'}
```

[69]:
```
new_cursor.execute('''select iter.id AS pos, src.name
from temp_numbers iter, C src
where iter.id <= length(src.name) - length(replace(src.name, ',', '')) LIMIT 1;
 ↪''')
for View_C in new_cursor:
```

```
    print(View_C)
```

```
{'pos': 2, 'name': 'mo,larry,curly'}
```

[70]:
```
new_cursor.execute('''select iter.id AS pos, src.name AS name1,
    substring_index(src.name,',',iter.id) AS name2,
    substring_index(
    substring_index(src.name,',',iter.id),',',-1) AS name3
    from temp_numbers iter,
    C src
    where iter.id <= length(src.name) - length(replace(src.name,',','')) LIMIT␣
 ↪2;''')
for View_C in new_cursor:
    print(View_C)
```

```
{'pos': 2, 'name1': 'mo,larry,curly', 'name2': 'mo,larry', 'name3': 'larry'}
{'pos': 1, 'name1': 'mo,larry,curly', 'name2': 'mo', 'name3': 'mo'}
```

6.15 Parsing an IP Address

[71]:
```
new_cursor.execute('''select substring_index(substring_index(y.ip,'.',1),'.
 ↪',-1) a,
       substring_index(substring_index(y.ip,'.',2),'.',-1) b,
       substring_index(substring_index(y.ip,'.',3),'.',-1) c,
       substring_index(substring_index(y.ip,'.',4),'.',-1) d
from (select '92.111.0.2' as ip from temp_numbers LIMIT 1) y;
''')
for address in new_cursor:
    print(address)
```

```
{'a': '92', 'b': '111', 'c': '0', 'd': '2'}
```

6.16 Comparing Strings by Sound

[72]:
```
new_cursor.execute('''CREATE TABLE author_names (
    a_name VARCHAR(50));''')
for author in new_cursor:
    print(author)
```

[77]:
```
new_cursor.execute('''INSERT INTO author_names (a_name) VALUES
('Johnson'),
('Jonson'),
('Jonsen'),
('Jensen'),
('Johnsen'),
('Shakespeare'),
('Shakspear'),
('Shaekspir'),
('Shakespar');
```

```
''')

for author in new_cursor:
    print(author)
```

[78]:
```
new_cursor.execute('''select distinct an1.a_name as name1, an2.a_name as name2,
    SOUNDEX(an1.a_name) as Soundex_Name
    from author_names an1
    join author_names an2
    on (SOUNDEX(an1.a_name)=SOUNDEX(an2.a_name)
    and an1.a_name!=  an2.a_name);''')

for author in new_cursor:
    print(author)
```

```
{'name1': 'Johnsen', 'name2': 'Johnson', 'Soundex_Name': 'J525'}
{'name1': 'Jensen', 'name2': 'Johnson', 'Soundex_Name': 'J525'}
{'name1': 'Jonsen', 'name2': 'Johnson', 'Soundex_Name': 'J525'}
{'name1': 'Jonson', 'name2': 'Johnson', 'Soundex_Name': 'J525'}
{'name1': 'Johnsen', 'name2': 'Jonson', 'Soundex_Name': 'J525'}
{'name1': 'Jensen', 'name2': 'Jonson', 'Soundex_Name': 'J525'}
{'name1': 'Jonsen', 'name2': 'Jonson', 'Soundex_Name': 'J525'}
{'name1': 'Johnson', 'name2': 'Jonson', 'Soundex_Name': 'J525'}
{'name1': 'Johnsen', 'name2': 'Jonsen', 'Soundex_Name': 'J525'}
{'name1': 'Jensen', 'name2': 'Jonsen', 'Soundex_Name': 'J525'}
{'name1': 'Jonson', 'name2': 'Jonsen', 'Soundex_Name': 'J525'}
{'name1': 'Johnson', 'name2': 'Jonsen', 'Soundex_Name': 'J525'}
{'name1': 'Johnsen', 'name2': 'Jensen', 'Soundex_Name': 'J525'}
{'name1': 'Jonsen', 'name2': 'Jensen', 'Soundex_Name': 'J525'}
{'name1': 'Jonson', 'name2': 'Jensen', 'Soundex_Name': 'J525'}
{'name1': 'Johnson', 'name2': 'Jensen', 'Soundex_Name': 'J525'}
{'name1': 'Jensen', 'name2': 'Johnsen', 'Soundex_Name': 'J525'}
{'name1': 'Jonsen', 'name2': 'Johnsen', 'Soundex_Name': 'J525'}
{'name1': 'Jonson', 'name2': 'Johnsen', 'Soundex_Name': 'J525'}
{'name1': 'Johnson', 'name2': 'Johnsen', 'Soundex_Name': 'J525'}
{'name1': 'Shakespar', 'name2': 'Shakespeare', 'Soundex_Name': 'S160'}
{'name1': 'Shaekspir', 'name2': 'Shakespeare', 'Soundex_Name': 'S160'}
{'name1': 'Shakspear', 'name2': 'Shakespeare', 'Soundex_Name': 'S160'}
{'name1': 'Shakespar', 'name2': 'Shakspear', 'Soundex_Name': 'S160'}
{'name1': 'Shaekspir', 'name2': 'Shakspear', 'Soundex_Name': 'S160'}
{'name1': 'Shakespeare', 'name2': 'Shakspear', 'Soundex_Name': 'S160'}
{'name1': 'Shakespar', 'name2': 'Shaekspir', 'Soundex_Name': 'S160'}
{'name1': 'Shakspear', 'name2': 'Shaekspir', 'Soundex_Name': 'S160'}
{'name1': 'Shakespeare', 'name2': 'Shaekspir', 'Soundex_Name': 'S160'}
{'name1': 'Shaekspir', 'name2': 'Shakespar', 'Soundex_Name': 'S160'}
{'name1': 'Shakspear', 'name2': 'Shakespar', 'Soundex_Name': 'S160'}
{'name1': 'Shakespeare', 'name2': 'Shakespar', 'Soundex_Name': 'S160'}
```

1.2.2 Numbers

[82]:
```
# cursor object for sakila database
mydb = mysql.connector.connect(**config)
my_cursor = mydb.cursor(dictionary=True)
```

[83]:
```
my_cursor.execute('''SELECT AVG(amount) AS avg_amount FROM payment''')
avg_res = my_cursor.fetchall()
print(avg_res)
```

```
[{'avg_amount': Decimal('4.201356')}]
```

[84]:
```
my_cursor.execute('''SELECT AVG(amount) AS avg_amount FROM payment GROUP BY␣
 ↪payment_date LIMIT 5''')
avg_res = my_cursor.fetchall()
print(avg_res)
```

```
[{'avg_amount': Decimal('2.990000')}, {'avg_amount': Decimal('0.990000')},
{'avg_amount': Decimal('5.990000')}, {'avg_amount': Decimal('0.990000')},
{'avg_amount': Decimal('9.990000')}]
```

[85]:
```
my_cursor.execute(''' SELECT MAX(amount) AS MAX_PAYMENT FROM payment ''')
minmax_res = my_cursor.fetchall()
print(minmax_res)
```

```
[{'MAX_PAYMENT': Decimal('11.99')}]
```

7.3 Summing the values in a column

[86]:
```
my_cursor.execute(''' SELECT SUM(amount) AS TOTAL_PAYMENT FROM payment ''')
minmax_res = my_cursor.fetchall()
print(minmax_res)
```

```
[{'TOTAL_PAYMENT': Decimal('67406.56')}]
```

7.4 Counting rows in a table

[87]:
```
my_cursor.execute(''' SELECT COUNT(*) AS total FROM film ''')
total_rows = my_cursor.fetchall()
print(total_rows)
```

```
[{'total': 1000}]
```

Computing avg without high and low values.

[88]:
```
# This will exclude all the Max and Min values (if the column contains more␣
 ↪than one max and min)
my_cursor.execute(''' SELECT AVG(amount) AS avg_without_high_low_values FROM␣
 ↪payment WHERE amount NOT IN ((SELECT MAX(amount) FROM payment), (SELECT␣
 ↪MIN(amount) FROM payment)) ''')
```

```
avg_w = my_cursor.fetchall()
print(avg_w)
```

```
[{'avg_without_high_low_values': Decimal('4.202790')}]
```

[89]:
```
# Use this to exclude only one set of Max and Min
my_cursor.execute(''' SELECT (SUM(amount) - MIN(amount) - MAX(amount)) /␣
 ↪(COUNT(*) - 2) AS avg_wmm FROM payment''')
avg_without_max_min = my_cursor.fetchall()
print(avg_without_max_min)
```

```
[{'avg_wmm': Decimal('4.201133')}]
```

This is the sample practice notebook. Please refer the SQL cookbook to continue with the practice. Book has the statistical example queries. 7.6 Generating a Running: Run the queries from 7.6 in sqlite as these queries use window functions which is not supported by MySQL Workbench

[90]:
```
import sqlite3
import pandas as pd
```

[91]:
```
conn = sqlite3.connect('week1.db')
c = conn.cursor()
```

[95]:
```
read_emp = pd.read_csv(r'D:/ADT/week1/emp.csv')
read_emp.to_sql('emp', conn, if_exists='replace', index = False) # Insert the␣
 ↪values from the csv file into the table 'emp'
```

[95]: 14

[96]:
```
read_dept = pd.read_csv(r'D:/ADT/week1/dept.csv')
read_dept.to_sql('dept', conn, if_exists='replace', index = False) # Insert the␣
 ↪values from the csv file into the table 'dept'
```

[96]: 4

7.6 Generating a Running Total

[97]:
```
c.execute('''
select ename, sal,
sum(sal) over (order by sal,empno) as running_total
from emp
order by 2;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])
```

```
df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[97]:

	ENAME	SAL	running_total
0	SMITH	800	800
1	JAMES	950	1750
2	ADAMS	1100	2850
3	WARD	1250	4100
4	MARTIN	1250	5350
5	MILLER	1300	6650
6	TURNER	1500	8150
7	ALLEN	1600	9750
8	CLARK	2450	12200
9	BLAKE	2850	15050
10	JONES	2975	18025
11	SCOTT	3000	21025
12	FORD	3000	24025
13	KING	5000	29025

[98]:
```
c.execute('''
select empno, sal,
sum(sal)over(order by sal,empno) as running_total1,
sum(sal)over(order by sal) as running_total2
from emp
order by 2;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[98]:

	EMPNO	SAL	running_total1	running_total2
0	7369	800	800	800
1	7900	950	1750	1750
2	7876	1100	2850	2850
3	7521	1250	4100	5350
4	7654	1250	5350	5350
5	7934	1300	6650	6650
6	7844	1500	8150	8150
7	7499	1600	9750	9750
8	7782	2450	12200	12200
9	7698	2850	15050	15050

```
10    7566  2975          18025          18025
11    7788  3000          21025          24025
12    7902  3000          24025          24025
13    7839  5000          29025          29025
```

7.7 Generating a Running Product

[99]:
```
#if you are facing issues in exp and ln function, it is because of the version
 ↪of MySQL that you are using.
import math
```

[100]:
```
conn.create_function("exp", 1, math.exp)
conn.create_function("ln", 1, math.log)
c= conn.cursor()
```

[101]:
```
c.execute('''
select empno,ename,sal,
exp(sum(ln(sal))over(order by sal,empno)) as running_prod
from emp
where deptno = 10;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[101]:

	EMPNO	ENAME	SAL	running_prod
0	7934	MILLER	1300	1.300000e+03
1	7782	CLARK	2450	3.185000e+06
2	7839	KING	5000	1.592500e+10

7.8 Smoothing a Series of Values - it is okay if you skip

7.9 Calculating a Mode

[102]:
```
c.execute('''
select sal
from emp
where deptno = 20
order by sal;
''')

colnames = c.description
colnames_list = []
```

```
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[102]:

	SAL
0	800
1	1100
2	2975
3	3000
4	3000

[103]:
```
# Mode is 3000
c.execute('''
    select sal
    from (
    select sal,
     dense_rank()over( order by cnt desc) as rnk
    from (
    select sal, count(*) as cnt
    from emp
    where deptno = 20
    group by sal
    ) x
    ) y
    where rnk = 1;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[103]:

	sal
0	3000

[104]:
```
c.execute('''
select sal,
dense_rank()over(order by cnt desc) as rnk
from (
select sal,count(*) as cnt
from emp
where deptno = 20
```

```
group by sal
) x;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[104]:

	sal	rnk
0	3000	1
1	800	2
2	1100	2
3	2975	2

7.10 Calculating a Median

[105]:
```
c.execute('''
select sal
from emp
where deptno = 20
order by sal;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[105]:

	SAL
0	800
1	1100
2	2975
3	3000
4	3000

[106]:
```
c.execute('''
with rank_tab (sal, rank_sal) as
(
select sal, cume_dist() over (order by sal)
from emp
```

```
where deptno=20
),
inter as
(
select sal, rank_sal from rank_tab
where rank_sal>=0.5
union
select sal, rank_sal from rank_tab
where rank_sal<=0.5
)
select avg(sal) as MedianSal
from inter;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[106]:

	MedianSal
0	1968.75

7.11 Determining the Percentage of a Total

[107]:
```
c.execute('''
select *
from emp where deptno=10;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[107]:

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
0	7782	CLARK	MANAGER	7839.0	9-Jun-06	2450	None	10
1	7839	KING	PRESIDENT	NaN	17-Nov-06	5000	None	10
2	7934	MILLER	CLERK	7782.0	23-Jan-07	1300	None	10

[108]:
```
c.execute('''
```

```
select (CAST(sum(case when deptno = 10 then sal end) AS FLOAT)/CAST(sum(sal) AS␣
 ↪FLOAT)
)*100 as pct
from emp;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[108]:
```
        pct
0  30.146425
```

[109]:
```
c.execute('''
select sum(case when deptno = 10 then sal end) as d10,
sum(sal)
from emp;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[109]:
```
    d10  sum(sal)
0  8750     29025
```

[110]:
```
c.execute('''
select (cast(
sum(case when deptno = 10 then sal end)
as FLOAT))/CAST(sum(sal) AS FLOAT
)*100 as pct
from emp;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])
```

```
df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[110]:

	pct
0	30.146425

[111]:
```
c.execute('''
select deptno,
sum(sal)over() total,
sum(sal)over(partition by deptno) d10
from emp;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[111]:

	DEPTNO	total	d10
0	10	29025	8750
1	10	29025	8750
2	10	29025	8750
3	20	29025	10875
4	20	29025	10875
5	20	29025	10875
6	20	29025	10875
7	20	29025	10875
8	30	29025	9400
9	30	29025	9400
10	30	29025	9400
11	30	29025	9400
12	30	29025	9400
13	30	29025	9400

[112]:
```
c.execute('''
select deptno,
sum(sal)over() total,
sum(sal)over(partition by deptno) d10
from emp
where deptno=10;
''')

colnames = c.description
colnames_list = []
```

```
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[112]:

	DEPTNO	total	d10
0	10	8750	8750
1	10	8750	8750
2	10	8750	8750

7.12 Aggregating Nullable Columns

[113]:
```
c.execute('''
select avg(coalesce(comm,0)) as avg_comm
from emp
where deptno=30;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[113]:

	avg_comm
0	366.666667

[114]:
```
c.execute('''
select avg(comm)
from emp
where deptno=30;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[114]:

	avg(comm)
0	550.0

[115]:
```
c.execute('''
select ename, comm
from emp
where deptno=30
order by comm desc;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[115]:

	ENAME	COMM
0	MARTIN	1400.0
1	WARD	500.0
2	ALLEN	300.0
3	TURNER	0.0
4	BLAKE	NaN
5	JAMES	NaN

7.13 Computing Averages Without High and Low Values

[116]:
```
c.execute('''
select avg(sal)
from emp
where sal not in (
(select min(sal) from emp),
(select max(sal) from emp)
);
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[116]:

	avg(sal)
0	1935.416667

[117]:
```
c.execute('''
select (sum(sal)-min(sal)-max(sal))/(count(*)-2) AS cnt
```

```
from emp;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[117]:

	cnt
0	1935

[118]:
```
c.execute('''
select sal, min(sal)over() min_sal, max(sal)over() max_sal
from emp;
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[118]:

	SAL	min_sal	max_sal
0	800	800	5000
1	1600	800	5000
2	1250	800	5000
3	2975	800	5000
4	1250	800	5000
5	2850	800	5000
6	2450	800	5000
7	3000	800	5000
8	5000	800	5000
9	1500	800	5000
10	1100	800	5000
11	950	800	5000
12	3000	800	5000
13	1300	800	5000

Skip 7.14, 7.15 as it uses traslate function

7.16 Finding Outliers Using the Median Absolute Deviation

[119]:
```
c.execute('''
    with rank_tab (sal, rank_sal) as (
    select sal, cume_dist() over (order by sal)
    from emp),
    inter as
    (
    select sal, rank_sal from rank_tab
    where rank_sal>=0.5
    union
    select sal, rank_sal from rank_tab
    where rank_sal<=0.5
    )
    ,
    medianSal (medianSal) as
    (
    select (max(sal)+min(sal))/2
    from inter),
    deviationSal (Sal,deviationSal) as
    (select Sal,abs(sal-medianSal)
    from emp join medianSal
    on 1=1
    )
    ,
    distDevSal (sal,deviationSal,distDeviationSal) as
    (
    select sal,deviationSal,cume_dist() over (order by deviationSal)
    from deviationSal
    ),
    DevInter (DevInter, sal) as
    (
    select min(deviationSal), sal
    from distDevSal
    where distDeviationSal >= 0.5
    union
    select max(DeviationSal), sal
    from distDevSal
    where distDeviationSal <= 0.5
    ),
    MAD (MedianAbsoluteDeviance) as
    (
    select abs(emp.sal-(min(devInter)+max(devInter))/2)
    from emp join DevInter on 1=1
    )
    select emp.sal,MedianAbsoluteDeviance,
    (emp.sal-deviationSal)/MedianAbsoluteDeviance as ratio
    from (emp join MAD on 1=1)
    join deviationSal on emp.sal=deviationSal.sal;
```

```
''')

colnames = c.description
colnames_list = []
for row in colnames:
    colnames_list.append(row[0])

df = pd.DataFrame(c.fetchall(), columns=colnames_list)
df
```

[119]:

	SAL	MedianAbsoluteDeviance	ratio
0	800	600	-2
1	1600	600	0
2	1250	600	0
3	1250	600	0
4	2975	600	4
5	1250	600	0
6	1250	600	0
7	2850	600	4
8	2450	600	3
9	3000	600	4
10	3000	600	4
11	5000	600	4
12	1500	600	0
13	1100	600	-1
14	950	600	-1
15	3000	600	4
16	3000	600	4
17	1300	600	0

Skip 7.17 Finding Anomalies Using Benford's Law

[120]:
```
conn.close()
```